

```
<!--Instituto Profesional DUOC UC-->  
<!--Profesor: Miguel-->
```

Desarrollo Full Stack III {

```
<Por="Hawk Durant","Rodrigo  
Candia","Emilio Jaramillo"/>
```

```
}
```



AGENDA

01

Introducción

02

Arquitectura

03

Stack tecnologico

04

Persistencia

05

Comunicación

06

Pruebas unitarias

07

Patrones de diseño

Recordemos . . . {



Municipalidad Valle del Sol

Gestión y prevención de emergencias

Objetivos del Sistema

Exponer una API REST coherente con roles (ciudadano, operador, emergencia).

Permitir reportes con geocodificación automática mediante APIs externas.

Coordinar la creación de emergencias, asignación de recursos y notificaciones.

Garantizar resiliencia ante fallos en servicios externos.

}

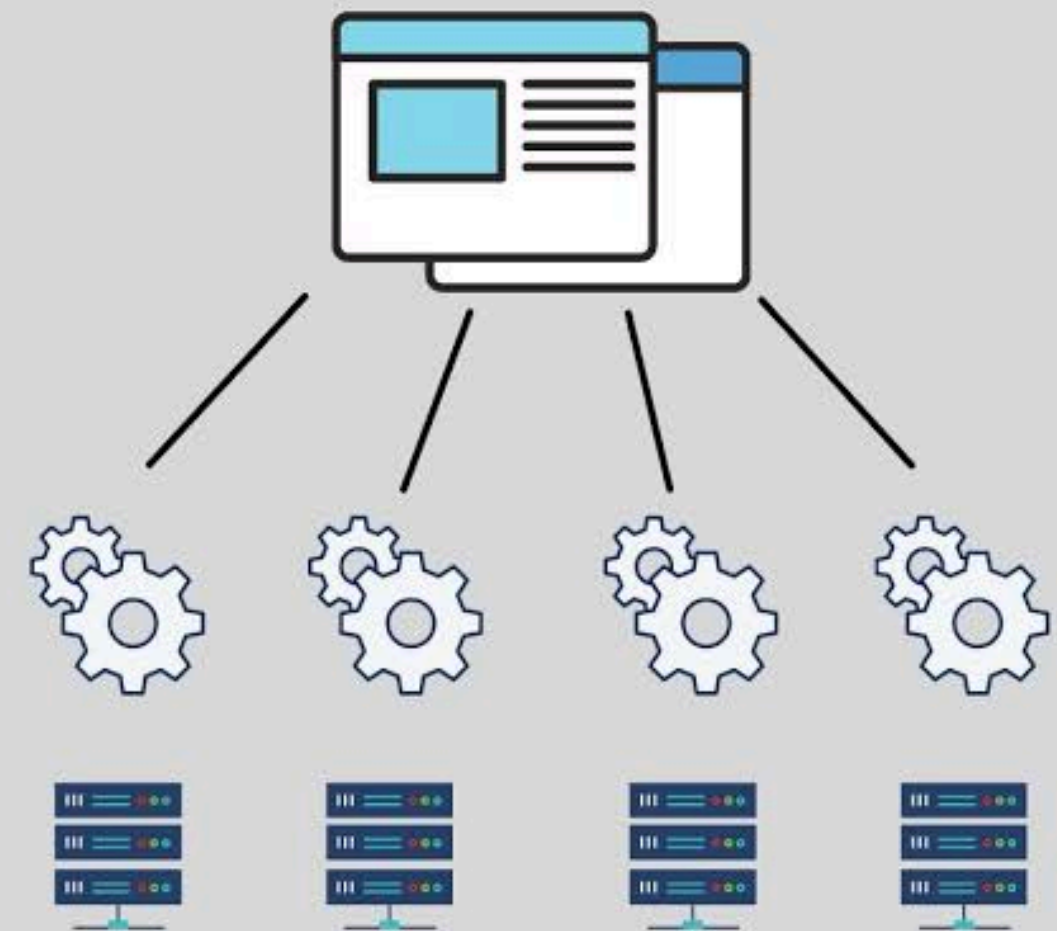
Arquitectura {

Arquitectura de Microservicios

Aislamiento de fallos: Un error en un módulo no tumba todo el sistema

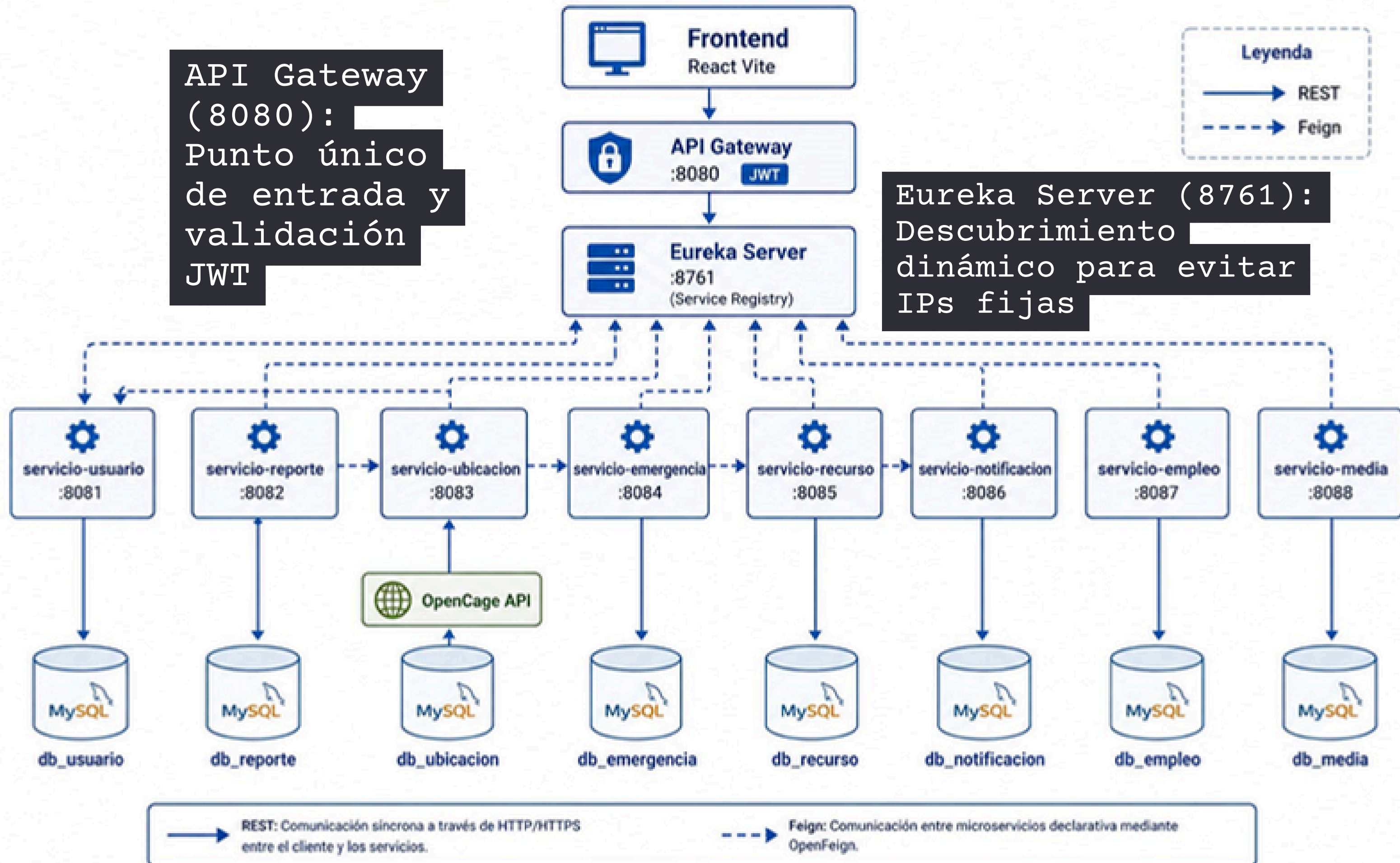
Evolución independiente: Cada módulo (usuarios, reportes, empleos) puede escalarse y desplegarse por separado

Modularidad: División clara de fronteras entre dominios de negocio (8 microservicios independientes)



}

S.I.G.I - Arquitectura de Microservicios - Valle del Sol



Stack Tecnológico {

01

Frontend:
React 19,
Vite y
Tailwind
CSS.

02

Backend:
Java 17 con
Spring Boot
3.5

03

Infraestructura:
Spring Cloud
(Gateway,
Eureka,
OpenFeign)

04

Base de
Datos: MySQL
8.0
gestionado
mediante
Docker
Compose

05

Integración Externa: API de
OpenCage para transformar
direcciones en coordenadas

}

Persistencia: Database per Service{

Enfoque:

Cada microservicio es dueño de su propio esquema lógico (db_usuario, db_reporte, etc.)

```
CREATE DATABASE IF NOT EXISTS db_usuario;  
CREATE DATABASE IF NOT EXISTS db_reporte;  
CREATE DATABASE IF NOT EXISTS db_ubicacion;  
CREATE DATABASE IF NOT EXISTS db_emergencia;  
CREATE DATABASE IF NOT EXISTS db_recurso;  
CREATE DATABASE IF NOT EXISTS db_notificacion;  
CREATE DATABASE IF NOT EXISTS db_empleo;  
CREATE DATABASE IF NOT EXISTS db_media;
```

Implementación:

Spring Data JPA: Manejo de datos mediante entidades anotadas y repositorios.

Sin Procedimientos Almacenados: Se priorizó JPA para facilitar las pruebas unitarias con Mocks.

DDL-Auto Update: Hibernate actualiza automáticamente las tablas según las clases Java en desarrollo.

}

Comunicación y Resiliencia {

Comunicación Interna:

Síncrona mediante OpenFeign para llamadas declarativas entre servicios.

Tolerancia a fallos:

Implementación de Circuit Breaker (Resilience4j)

Ejemplo de Fallback:

Si la API de OpenCage falla, el servicio de ubicación activa un flujo de respaldo para permitir que el reporte se cree sin coordenadas, evitando el bloqueo del usuario

```
package com.sigi.servicio_reporte.service;

import org.springframework.stereotype.Service;

import com.sigi.servicio_reporte.client.UbicacionClient;
import com.sigi.servicio_reporte.client.dto.CoordenadasDto;

import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;

/**
 * Capa fina para envolver Feign con Circuit Breaker (Resilience4j).
 * Si ubicacion falla, el fallback devuelve coordenadas nulas y el reporte ig
 */
@Service
public class UbicacionConsultaService {

    private final UbicacionClient ubicacionClient;

    public UbicacionConsultaService(UbicacionClient ubicacionClient) {
        this.ubicacionClient = ubicacionClient;
    }
}
```

}

Aseguramiento de Calidad y Pruebas{

METODOLOGIA

Pruebas unitarias sobre
la capa de servicio
usando JUnit 5 y
Mockito

MÉTRICA LOGRADA

92,4% de cobertura global de
líneas de código

The screenshot shows the VS Code interface with the 'TERMINAL' tab active. It displays the output of running tests and a detailed code coverage report. The test results show 1 failed, 8 passed, and 9 total test suites. The coverage report lists various files and their corresponding metrics for statements, branches, functions, and lines.

Test Suites: 1 failed, 8 passed, 9 total
Tests: 1 failed, 17 passed, 18 total
Snapshots: 0 total
Time: 10.071 s
Ran all test suites.

Code Coverage Report:

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	71.26	59.66	72.54	70.9	
components	100	56.25	100	100	
AuthHero.jsx	100	100	100	100	
...ncendioCard.jsx	100	56.25	100	100	21-41
constants	87.5	90	85.71	92.85	
usuariosPrueba.js	87.5	90	85.71	92.85	152
context	86.95	75	87.5	85.71	
AuthContext.jsx	86.95	75	87.5	85.71	45-46,71
pages	98.3	85.71	100	98.24	
Login.jsx	100	90	100	100	54
Register.jsx	97.5	84	100	97.43	50
services	31.25	27.5	33.33	29.5	
apiClient.js	28.57	22.85	50	25.64	10,23-71,81-92
authService.js	0	100	0	0	4-33
empleoService.js	25	100	25	25	8-30,38
mediaService.js	75	60	100	75	7,11
test	100	100	100	100	
apiMock.js	100	100	100	100	
utils	100	60	100	100	
reporteMappers.js	100	60	100	100	10-21

Test Suites: 9 passed, 9 total
Tests: 18 passed, 18 total
Snapshots: 0 total
Time: 8.864 s
Ran all test suites.

Patrones de Diseño Aplicados {

Singleton:

Servicios de Spring con instancia única para consistencia (ej. utilidad JWT)

Repository:

Abstracción total de la persistencia de datos

DTO (Data Transfer Object):

Desacoplamiento entre las entidades de la base de datos y la respuesta de la API

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.sigi.servicio_emergencia.client.NotificacionFeignClient;
import com.sigi.servicio_emergencia.client.RecursoFeignClient;
import com.sigi.servicio_emergencia.client.dto.AlertaEmergenciaFeignRequest;
import com.sigi.servicio_emergencia.client.dto.AsignacionRecursoFeignRequest;
import com.sigi.servicio_emergencia.dto.ActualizarEstadoRequest;
import com.sigi.servicio_emergencia.dto.CrearEmergenciaRequest;
import com.sigi.servicio_emergencia.dto.EmergenciaResponse;
import com.sigi.servicio_emergencia.model.Emergencia;
import com.sigi.servicio_emergencia.model.Emergencia.EstadoEmergencia;
import com.sigi.servicio_emergencia.model.Emergencia.PrioridadIncendio;
import com.sigi.servicio_emergencia.repository.EmergenciaRepository;

@Service
public class EmergenciaService {

    private static final Logger log = LoggerFactory.getLogger(EmergenciaService.class);

    private final EmergenciaRepository emergenciaRepository;
    private final RecursoFeignClient recursoFeignClient;
    private final NotificacionFeignClient notificacionFeignClient;

    ...
}
```

```
<!--Desarrollo Full Stack III-->
```

Gracias! {

```
<Conclusion="Se logró integrar un  
ecosistema modular, escalable y resiliente  
que cumple con las demandas técnicas de una  
gestión municipal moderna"/>
```